CADT

IDT វិទ្យាស្ថានបច្ចេកវិទ្យាឌីជីថល
Institute of Digital Technology

Name: Red Bora G2
Lecturer: Yann Kimhouy

Assignment Database Week 5

Group Work

Exercise 5: Simple Google Calendar Event - Write a MySQL query to create database:

USER (user_id, name, nickname, email, password, phone_number, dob, gender)

EVENT (event_id, event_name, description, event_location, date, start_time, end_time, google_meet_link, notify_before)

PROFILE_IMAGE (image_id, image_url, user_id)

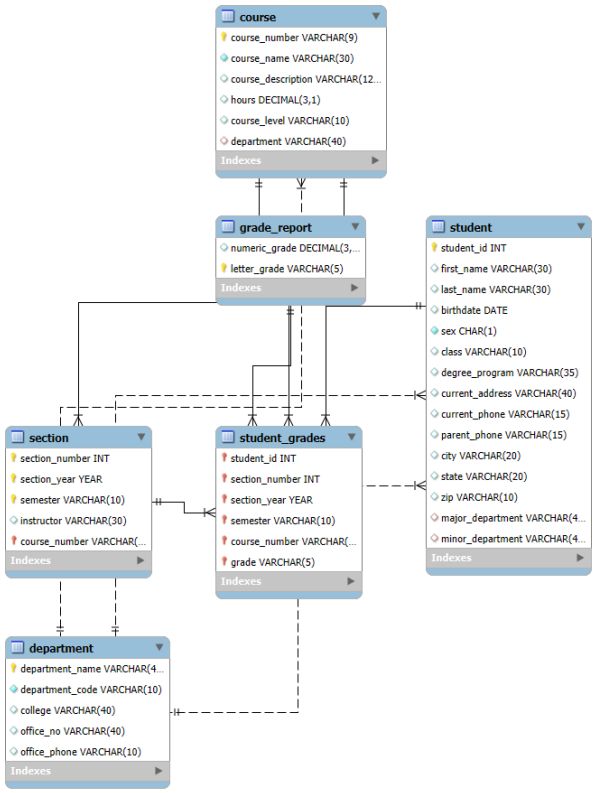
MySQL:

```
CREATE DATABASE google_calendar;

USE google_calendar;


CREATE TABLE USER (

   user_id INT AUTO_INCREMENT PRIMARY KEY,

   Username VARCHAR(100) NOT NULL,

   nickname VARCHAR(50),

   email VARCHAR(150) NOT NULL UNIQUE,

   User_password VARCHAR(255) NOT NULL,

   phone_number VARCHAR(20),

   dob DATE,

   gender ENUM('Male', 'Female', 'Other')

);



CREATE TABLE EVENT (

   event_id INT AUTO_INCREMENT PRIMARY KEY,

   user_id INT NOT NULL,

   event_name VARCHAR(150) NOT NULL, description TEXT,
```

```
    event_location VARCHAR(200),
    start_datetime DATE NOT NULL,
    end_datetime DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    google_meet_link VARCHAR(255),
    notify_before INT,

    FOREIGN KEY (user_id) REFERENCES USER(user_id),

    CHECK (end_datetime >= start_datetime)
);
CREATE TABLE PROFILE_IMAGE (
    image_id INT AUTO_INCREMENT PRIMARY KEY,
    image_url VARCHAR(255) NOT NULL,
    user_id INT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES USER(user_id)
);

INSERT INTO USER (name, nickname, email, password, phone_number, dob, gender)
VALUES
('John Smith', 'johnny', 'john@example.com', 'password123', '1234567890', '2005-06-15', 'Male'),
('Emily Davis', 'em', 'emily@example.com', 'mypassword456', '0987654321', '2006-02-20', 'Female');

INSERT INTO EVENT
(user_id, event_name, description, event_location, start_datetime, end_datetime, start_time, end_time, google_meet_link, notify_before)
VALUES
(1, 'Math Study Group', 'Prepare for algebra test', 'Library Room 2', '2026-03-01', '2026-03-01', '15:00:00', '17:00:00', 'https://meet.google.com/abc-xyz', 30),
(2, 'Birthday Party', 'Emily’s 20th Birthday Celebration', 'Emily’s House', '2026-03-05', '2026-03-05', '18:00:00', '22:00:00', NULL, 60);
```

```
INSERT INTO PROFILE_IMAGE (image_url, user_id)
VALUES
('https://example.com/images/john.jpg', 1),
('https://example.com/images/emily.jpg', 2);

ALTER TABLE EVENT
ADD CONSTRAINT chk_time
CHECK (end_time > start_time);

UPDATE EVENT
SET end_time = '20:00:00'
WHERE event_id = 1;

SELECT * FROM USER;
SELECT * FROM EVENT;
SELECT * FROM PROFILE_IMAGE;
```

Relational Schema:



HomeWork

1. Consider the following set of requirements for a UNIVERSITY database that is used to keep track of students' transcripts.

   -The university keeps track of each student's name, student number, social security number, current address and phone number, permanent address and phone number, birth date, sex, class (freshman, sophomore, ., graduate), major department, minor department (if any), and degree program (BA, BS, PHD). Some user applications need to refer to the city, state, and ZIP codes of the student's permanent address and the student's last name. Both social security number and student number have unique value for each student.

   -Each department is described by a name, department code, office number, office phone number, and college. Both name and code have unique values for each department.

   -Each course has a course name, description, course number, number of semester hours, level, and offering department. The value of the course number is unique for each course.

   -Each section has an instructor, semester, year, course, and section number. The section number distinguishes sections of the same course that are taught during the same semester/year; its values are 1, 2, 3, . up to the number of sections taught during each semester.

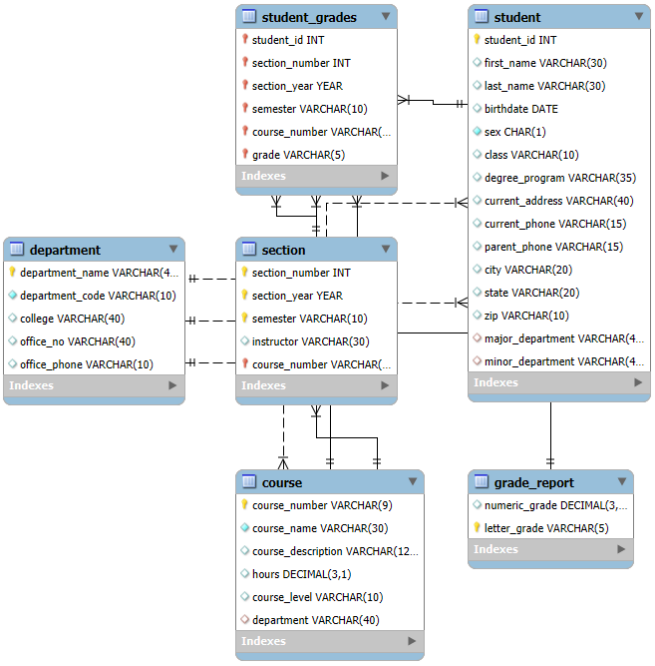
   -A Grade report has a student, section, letter grade, and numeric grade (0, 1, 2, 3, or 4

MySQL:

```
CREATE DATABASE IF NOT EXISTS UNIVERSITY;
USE UNIVERSITY;

CREATE TABLE DEPARTMENT (
   DepartmentCode VARCHAR(10) PRIMARY KEY,
   Name VARCHAR(100) UNIQUE NOT NULL,
   OfficeNumber VARCHAR(20),
   OfficePhone VARCHAR(20),
   College VARCHAR(100)
);

CREATE TABLE STUDENT (
   StudentNumber INT PRIMARY KEY,
   SSN CHAR(11) UNIQUE NOT NULL,
   FirstName VARCHAR(50) NOT NULL,
   LastName VARCHAR(50) NOT NULL,
   CurrentAddress TEXT,
   CurrentPhone VARCHAR(20),
   PermanentAddress TEXT,
```

```
    PermanentCity VARCHAR(50),
    PermanentState VARCHAR(50),
    PermanentZip VARCHAR(10),
    PermanentPhone VARCHAR(20),
    BirthDate DATE,
    Sex CHAR(1),
    Class ENUM('Freshman', 'Sophomore', 'Junior', 'Senior', 'Graduate'),
    MajorDeptCode VARCHAR(10),
    MinorDeptCode VARCHAR(10),
    DegreeProgram ENUM('BA', 'BS', 'PHD'),
    FOREIGN KEY (MajorDeptCode) REFERENCES DEPARTMENT(DepartmentCode),
    FOREIGN KEY (MinorDeptCode) REFERENCES DEPARTMENT(DepartmentCode),
    CONSTRAINT chk_sex CHECK (Sex IN ('M', 'F', 'O'))
);

CREATE TABLE COURSE (
    CourseNumber VARCHAR(20) PRIMARY KEY,
    CourseName VARCHAR(100) NOT NULL,
    Description TEXT,
    SemesterHours INT,
    Level VARCHAR(20),
    OfferingDeptCode VARCHAR(10),
    FOREIGN KEY (OfferingDeptCode) REFERENCES DEPARTMENT(DepartmentCode),
    CONSTRAINT chk_hours CHECK (SemesterHours > 0)
);

CREATE TABLE SECTION (
    SectionID INT AUTO_INCREMENT PRIMARY KEY,
    SectionNumber INT NOT NULL,
    Instructor VARCHAR(100),
    Semester VARCHAR(20),
    Year INT,
    CourseNumber VARCHAR(20),
    FOREIGN KEY (CourseNumber) REFERENCES COURSE(CourseNumber),
    CONSTRAINT chk_year CHECK (Year >= 1900)
);

CREATE TABLE GRADE_REPORT (
    StudentNumber INT,
    SectionID INT,
    Grade CHAR(2),
    PRIMARY KEY (StudentNumber, SectionID),
    FOREIGN KEY (StudentNumber) REFERENCES STUDENT(StudentNumber) ON DELETE CASCADE,
    FOREIGN KEY (SectionID) REFERENCES SECTION(SectionID) ON DELETE CASCADE,
    CONSTRAINT chk_grade CHECK (Grade IN ('A', 'B', 'C', 'D', 'F', 'I', 'W', 'P'))
);
```

Relation Schema:



2. Consider a MAIL_ORDER database in which employees take orders for parts from customers. The data requirements are summarized as follows:

   -The mail order company has employees, each identified by a unique employee number, first and last name, and zip code.

   -Each customer of the company is identified by a unique customer number, first and last name, and zip code.

   -Each part sold by the company is identified by a unique part number, a part name, price, and quantity in stock.

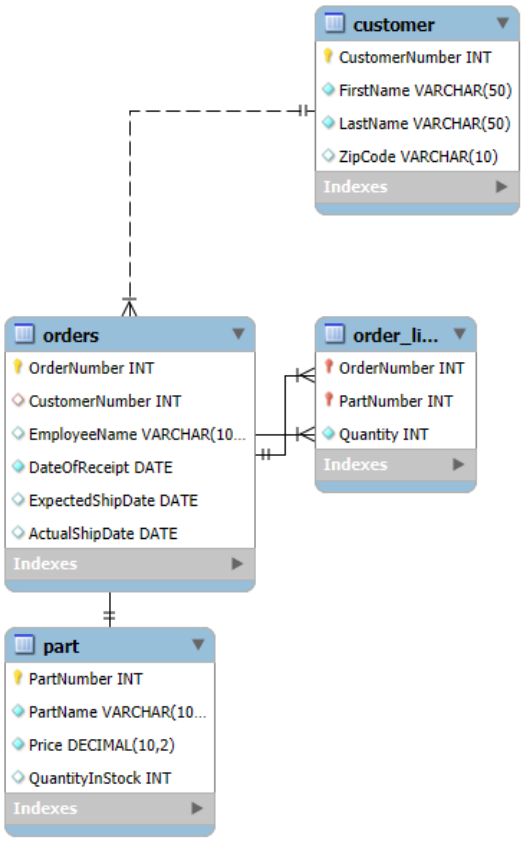
   -Each order placed by a customer is taken by an employee and is given a unique order number. Each order contains specified quantities of one or more parts. Each order has a date of receipt as well as an expected ship date. The actual ship data is also recorded.

MySQL:

```
CREATE DATABASE IF NOT EXISTS MAIL_ORDER;
USE MAIL_ORDER;

CREATE TABLE CUSTOMER (
   CustomerNumber INT PRIMARY KEY,
   FirstName VARCHAR(50) NOT NULL,
   LastName VARCHAR(50) NOT NULL,
   ZipCode VARCHAR(10),
```

```
    CONSTRAINT chk_zip CHECK (LENGTH(ZipCode) >= 5)
);

CREATE TABLE PART (
    PartNumber INT PRIMARY KEY,
    PartName VARCHAR(100) NOT NULL,
    Price DECIMAL(10, 2) NOT NULL,
    QuantityInStock INT DEFAULT 0,
    CONSTRAINT chk_price CHECK (Price > 0),
    CONSTRAINT chk_stock CHECK (QuantityInStock >= 0)
);

CREATE TABLE ORDERS (
    OrderNumber INT PRIMARY KEY,
    CustomerNumber INT NOT NULL,
    EmployeeName VARCHAR(100),
    DateOfReceipt DATE NOT NULL,
    ExpectedShipDate DATE,
    ActualShipDate DATE,
    FOREIGN KEY (CustomerNumber) REFERENCES CUSTOMER(CustomerNumber),
    CONSTRAINT chk_dates CHECK (ExpectedShipDate >= DateOfReceipt),
    CONSTRAINT chk_actual_ship CHECK (ActualShipDate >= DateOfReceipt)
);



CREATE TABLE ORDER_LINE (
    OrderNumber INT,
    PartNumber INT,
    Quantity INT NOT NULL,
    PRIMARY KEY (OrderNumber, PartNumber),
    FOREIGN KEY (OrderNumber) REFERENCES ORDERS(OrderNumber) ON DELETE CASCADE,
    FOREIGN KEY (PartNumber) REFERENCES PART(PartNumber),
    CONSTRAINT chk_quantity CHECK (Quantity > 0)
);
```

Relation Schema:


customer
CustomerNumber INT
FirstName VARCHAR(50)
LastName VARCHAR(50)
ZipCode VARCHAR(10)
Indexes
orders
OrderNumber INT
CustomerNumber INT
EmployeeName VARCHAR(10...
DateOfReceipt DATE
ExpectedShipDate DATE
ActualShipDate DATE
Indexes
order_li...
OrderNumber INT
PartNumber INT
Quantity INT
Indexes
part
PartNumber INT
PartName VARCHAR(10...
Price DECIMAL(10,2)
QuantityInStock INT
Indexes